

B#

Compiling a closed-world subset of MSBuild to a native build host

Simon Rozsival · research playground · proof-of-concept

The problem: we re-pay for MSBuild's dynamism every build

Even a trivial `net11.0` console app, **every** invocation:

- Re-evaluates the whole shape — project + full SDK closure
- **~500 targets · 199 UsingTasks · 14 assemblies** — for Hello World
- Dynamic dicts · reflection-loaded tasks · runtime topo-sort
- CoreCLR + JIT cold start

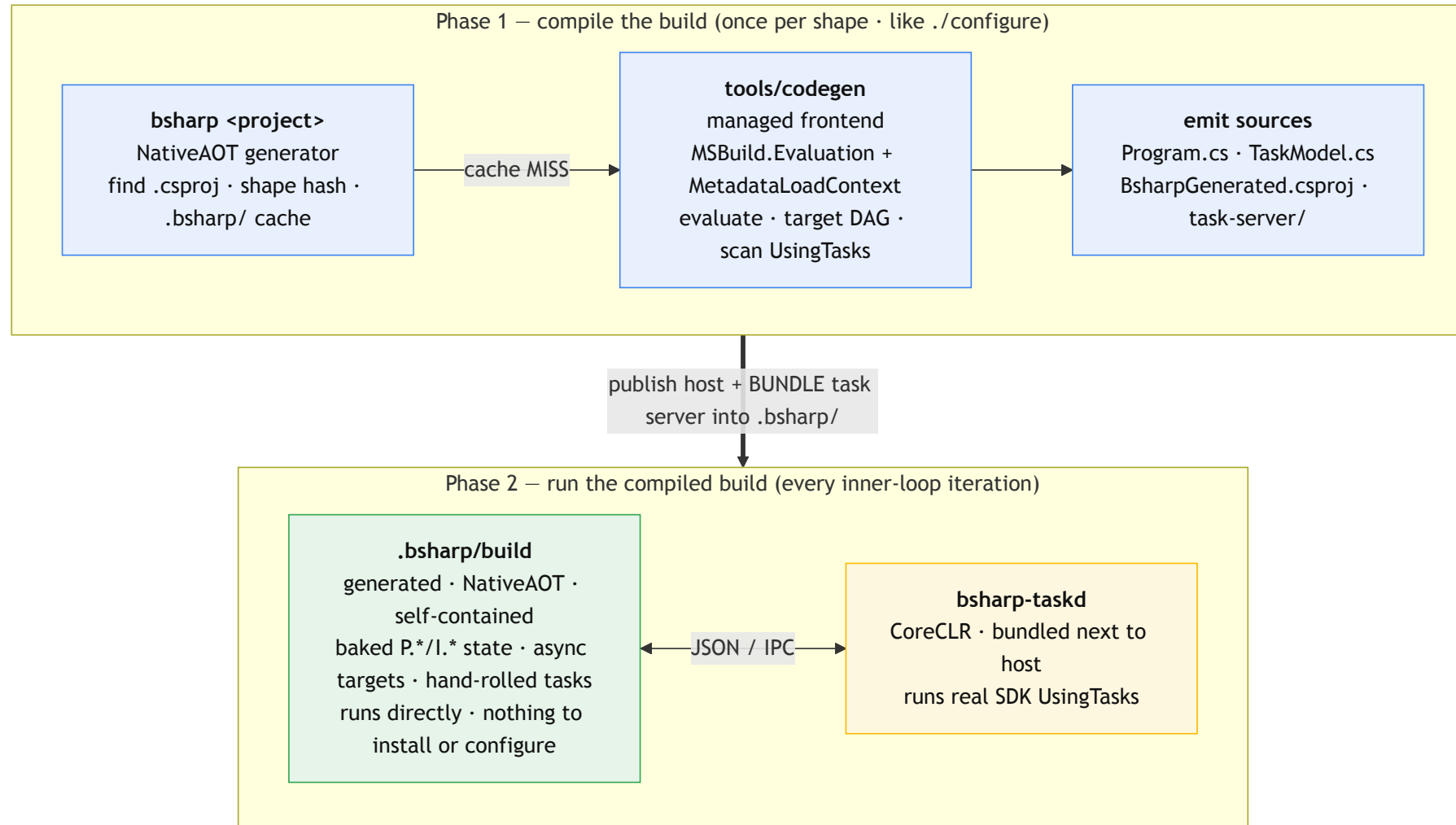
The hypothesis: the *shape* is a compile-time constant

- Inner loop: **source** changes constantly, **shape** almost never
- So evaluation + DAG + task binding = **redundant work**

The experiment: evaluate the shape once → **code-generate specialized C#**
→ **NativeAOT-compile** a per-project build binary.

A toy, not a product. How far does it get before the real world breaks it?

Architecture: compile the build, then run it (two phases)



Shape hash: when do you re-compile the build?

The generator hashes the **shape**:

- `.csproj` · imported `.props` / `.targets` · `ProjectReference` s
- `Directory.Build.*` · `Directory.Packages.props` · `global.json`
- lock / assets files · `-p:X=Y` globals

Source edit → run `.bsharp/build` . Shape edit → regenerate.

Codegen: baked state + targets as methods

- Properties → **typed static fields** (`P.Configuration = "Debug"`)
- Items → **typed lists** (`I.*`)
- Target → **async ValueTask** + execute-once guard; deps expanded at codegen
- Dynamic `Targets.Run(string)` dispatcher → often **never emitted**
- **Fast no-op**: timestamp check, returns before any target runs

Tasks: the NativeAOT / CoreCLR hybrid

- **Structural tasks** → compiled into the host (no IPC)

Copy · Delete · WriteLinesToFile · Message · Exec · Csc -wrapper ...

- **Real SDK UsingTask s** → **CoreCLR task server** over JSON/IPC

Csc · ResolvePackageAssets · GenerateDepsFile ...

NativeAOT can't load task assemblies dynamically → isolate that in CoreCLR, keep the host AOT-clean and fast to start.

● LIVE DEMO — setup

```
demo-cd                # fixtures/console-net11
cat console-net11.csproj # plain SDK console app
cat Program.cs          # Hello 9

ls .bsharp/             # generated host + BUNDLED task server
ls .bsharp/bin/.../publish/ | grep bsharp-taskd # self-contained
wc -l .bsharp/Program.cs # ~42k lines of specialized C#
head .bsharp/tasks.report.txt # 199 UsingTasks, 14 assemblies
```


● LIVE DEMO — an early timing, for one trivial app

```
echo "== dotnet =="; t-dotnet      # ~1.0 s warm no-op (--no-restore)
echo "== .bsharp/build =="; t-build # ~57 ms – the compiled build, run directly

# incremental edit
sed -i '' 's/Hello 9/Hello, B#!/' Program.cs
time ./bsharp/build --no-restore -v:quiet build    # ~150 ms vs dotnet ~990 ms
./bsharp/build run -v:quiet                       # Hello, B#!
```

no-op	dotnet <code>--no-restore</code> ~1.0 s · <code>.bsharp/build</code> ~57 ms (~18×)
incremental	dotnet ~0.99 s · <code>.bsharp/build</code> ~150 ms (~6.7×)

Early timings — one trivial console app

median ms, ~20 runs. Baseline = `dotnet build` (restore on). **Not apples-to-apples.**

Scenario	<code>.bsharp/build</code> (best)	vs dotnet (no- restore)	vs dotnet (restore- on)
noop	57	1014 → ~18×	1534 → ~27×
incremental	138	994 → ~7×	1522 → ~11×
clean	342	—	1558 → ~4.5×
restore	245	—	1020 [†] → ~4.2×

[†] vs `dotnet restore`. Ratios come **entirely from doing less** · + one-time ~30–80 s publish.

● LIVE DEMO — reasoning about a shape

```
gen-bsharp audit | head -40
```

- Evaluates the project and reports the **shape** as JSON — targets, tasks, `CallTarget` / `<MSBuild>` sites, dynamic imports, `UsingTask` issues
- **No host generated** — this is the bring-up / triage tool for bigger SDKs

Be honest about the numbers

- **Not a fair fight, on purpose** — speedup is bought by **doing less**
- **Missing functionality?** maybe the build now skips something important — but it builds & runs, so maybe it wasn't important here in the first place?
- Broad across these scenarios ($\sim 4\text{--}27\times$) — but **only this one app**
- **Tradeoff:** compiled for a *fixed shape*; shape change $\rightarrow$ regenerate
- Fast-noop / fast-restore are timestamp-based + conservative
- One-time $\sim 30\text{--}80$ s NativeAOT publish per shape
- macOS noise $\rightarrow$ **median**, trust ratios not absolutes

Known limitations

- **Closed-world subset** — out-of-subset shapes rejected, never approximated
- Works today: **console apps + class libraries**, `net11.0`
- Not yet: **MAUI / Blazor**, `watch`, target batching, package-supplied targets
- macOS arm64 / .NET 11 preview only

Where this experiment is honestly at

Works: runnable binary for console apps + class libs, run directly as a self-contained `.bsharp/build`.

Open / hard:

- Shape-change detection on the hot path
- Shrink the one-time publish cost
- Broaden the subset — **MAUI is a distant north star, not a plan**
- Correctness oracle vs stock MSBuild (almost none today)

The ask: worth poking at further? "Probably not, because..." welcome.

Thanks — questions?

- Repo: `simonrozsival/bsharp` · gist:
<https://gist.github.com/simonrozsival/8f252e57a06dc2e11ae7a51c9a0473b2>
- `README.md` + `DESIGN.md` §0 = current design summary
- Try it: `./build.sh` → `bsharp build` once, then `./.bsharp/build run` in `fixtures/console-net11`